

CS4296 Cloud Computing

City University of Hong Kong

Assignment 3

Experiment Report

Student Name: Shek King Fung

Student ID: 58527549

1. Algorithm Choices and Explanation

In this report, I have chosen three common classification models stated in [1], [2]. They are Logistic Regression, Random Forest and Naive Bayes. The input for all models should be a 784-dimensional vector, so the column in the .csv file is merged into a single vector by using the VectorAssembler [3]. And SparkSession [4] is used in the program to access the Spark Application, which is the pyspark.ml here. The models in the pyspark.ml is trained by calling the fit(dataset) method and make predictions by calling the transform(dataset) method.

Logistic Regression is chosen as the primary algorithm as it is a simpler classification technique that is often used as the baseline for classification. The parameter impact will also be conducted in Logistic Regression.

Logistic Regression [5] is a linear classification algorithm that uses a SoftMax function to make predictions, which is the probability for the 10-digit classes here. To prevent over-fitting, regularization is required. Therefore, the StandardScaler [6] is used in the program, which standardizes features. The data are scales to unit deviation and zero mean (withStd=True, withMean=True) in the program for processing.

The parameters for the LogisticRegression model include *maxIter*, *regParam* and *family*. *maxIter* is the number of training iterations, and it is equal to 50 in this program; *regParam* is the regularization strength, and it is set to 0.01 here; *family* is the description of the label distribution to be used in the model, since there are 10 classes (10-digit) for the input data, it is set to multinomial here.

Random Forest [7] is an ensemble learning method that constructs multiple decision trees during training and outputs the majority vote across all trees. It used bagging and random subsets of features to prevent over-fitting. The Standard Scaler [6] is also used for Random Forest Classifier model, with the same setting in Logistic Regression.

The parameters for Random Forest Classifier model include *numTrees*, *maxDepth* and *seed*. *numTrees* is the number of trees and it is set to 50 here; *maxDepth* is maximum depth of each tree, which must be in range [0, 30] and is set to 10 here; *seed* is ensure the reproducibility since without it, the model will take subset of features randomly, it is set to 42 here.

Naive Bayes [8] is a high-bias/low-variance probabilistic classifier, which works well in a limited amount of data and doesn't tend to overfit data. Therefore, raw data will be used as the input without scale.

The parameters for Naive Bayes model include *smoothing* and *modelType*. *smoothing* is used to prevent zero probabilities and the default value 1.0 is used here; *modelType* is the type of model to used, since there are discrete features, it is set to multinomial here.

Since all three models are classification models, they all give the predicted digit as the output.

2. Program Test Result

The program is tested on AWS EMR version emr-6.15.0. This is because this EMR version has the NumPy library installed by default for the pyspark.ml to use. In the EMR, one Primary instance, one Core instance, and one Task instance are used. They are all r8g.xlarge type of instance. When trying to run the program in EMR's steps, please choose client mode to run the program to see the print results in stdout, which take around 2-3 minutes after steps completion to create. Using cluster mode will make the stdout unable to be viewed. The program takes the first parameter as path to the train data and second one as path to the test data. This needs to be provided when adding steps. The following is the test results for Logistic Regression.

ALGORITHM 1 - LOGISTIC REGRESSION

Training: Logistic Regression

Training time: 15.09 seconds

Training accuracy: 0.9261

Test accuracy: 0.9230

3. Parameter Impact

In this report, the parameters impact is only tested on the Logistic Regression model. Parameters *maxIter* and *regParam* are being tested. And the testing is done by using the MulticlassClassificationEvaluator [9], which is an evaluator that can be used on multiple classes. The results are as follows.

---Testing different maxIter values---

=====

Training: Logistic Regression (maxIter=10)

Training time: 4.56 seconds

Training accuracy: 0.9192

Test accuracy: 0.9181

=====

Training: Logistic Regression (maxIter=30)

Training time: 7.45 seconds

Training accuracy: 0.9258

Test accuracy: 0.9227

=====

Training: Logistic Regression (maxIter=50)

Training time: 9.33 seconds

Training accuracy: 0.9261

Test accuracy: 0.9230

=====

Training: Logistic Regression (maxIter=100)

Training time: 15.23 seconds

Training accuracy: 0.9264

Test accuracy: 0.9229

---Testing different regParam values---

=====

Training: Logistic Regression (regParam=0.000)

Training time: 8.89 seconds

Training accuracy: 0.9403

Test accuracy: 0.9244

=====

Training: Logistic Regression (regParam=0.001)

Training time: 8.83 seconds

Training accuracy: 0.9374

Test accuracy: 0.9264

=====

Training: Logistic Regression (regParam=0.010)

Training time: 8.88 seconds

Training accuracy: 0.9261

Test accuracy: 0.9230

=====

Training: Logistic Regression (regParam=0.100)

Training time: 8.80 seconds

Training accuracy: 0.9042

Test accuracy: 0.9077

=====

Training: Logistic Regression (regParam=1.000)

Training time: 7.77 seconds

Training accuracy: 0.8615

Test accuracy: 0.8688

Summary of Parameter Impact:

Logistic Regression Parameters	Training time	Training accuracy	Test accuracy
maxIter=10	4.56 seconds	0.9192	0.9181
maxIter=30	7.45 seconds	0.9258	0.9227
maxIter=50	9.33 seconds	0.9261	0.9230
maxIter=100	15.23 seconds	0.9264	0.9229
regParam=0.000	8.89 seconds	0.9403	0.9244
regParam=0.001	8.83 seconds	0.9374	0.9264
regParam=0.010	8.88 seconds	0.9261	0.9230
regParam=0.100	8.80 seconds	0.9042	0.9077
regParam=1.000	7.77 seconds	0.8615	0.8688

From the results, it could be discovered that for *maxIter*, it will take longer training time with larger *maxIter* value. And the training accuracy performs best when *maxIter* is equal to 100 (0.9264), and second best when *maxIter* is equal to 50 (0.9261). While the test accuracy performs best when *maxIter* = 50 (0.9230), and second best when *maxIter* = 100 (0.9229). Therefore, we could conclude that the *maxIter* doesn't always give a better result while increasing its value, and we also need to consider it taking more time with higher *maxIter*.

For *regParam*, it could be found a roughly pattern that the training time will be smaller when setting the *regParam* higher, but there is also exception. For example, when *regParam* is equal to 0.001 (8.83 seconds), the training time is smaller than the one when *regParam* is equal to 0.010 (8.88 seconds). For training accuracy, it performs better with a smaller *regParam*. However, for test accuracy, *regParam* = 0.001 (0.9264) performs better than when *regParam* = 0.000 (0.9244). Therefore, we could conclude that *regParam* should be a small value that are not equal to zero to give the best performance.

4. Algorithm Comparison

The program also considers Random Forest Classifier and Naive Bayes as alternative ML models for comparison. The following are their results.

ALGORITHM 2: RANDOM FOREST CLASSIFIER

Training: Random Forest

Training time: 22.87 seconds

Training accuracy: 0.9600

Test accuracy: 0.9463

ALGORITHM 3: NAIVE BAYES CLASSIFIER

Training: Naive Bayes

Training time: 1.99 seconds

Training accuracy: 0.8253

Test accuracy: 0.8365

The results summary of all three algorithms in the program:

Model	Training time	Training accuracy	Test accuracy
Logistic Regression	15.09 seconds	0.9261	0.9230
Random Forest Classifier	22.87 seconds	0.9600	0.9463
Naive Bayes	1.99 seconds	0.8253	0.8365

From the results summary, Naive Bayes model gives the fastest training time, which is 7 times faster than the other two models. However, it gives the poorest training accuracy and test accuracy in all models, which is only around 80%. The Logistic Regression also takes longer to train, with a training time of 15.09 seconds. But it gives the second-best training accuracy and test accuracy, with 92.61% and 92.3% respectively. The Random Forest Classifier is the most time-consuming one, with a training time of 22.87 seconds. But it also achieves the highest training accuracy and test accuracy, with 96% and 94.63% respectively.

In conclusion, when aiming for fastest performance with acceptable accuracy, Naïve Bayes should be used. While aiming for best accuracy, Random Forest Classification should be used. While aiming for accuracy-time balance, Logistic Regression should be used.

References

- [1] MathWorks. “Choosing the Best Machine Learning Classification model and avoiding overfitting,” MathWorks. [Online]. Available: <https://ww2.mathworks.cn/campaigns/offers/next/choosing-the-best-machine-learning-classification-model-and-avoiding-overfitting.html> [Accessed: Apr. 7, 2026].
- [2] Apache Spark. “Classification and regression - Spark 4.1.1 Documentation,” Apache Spark. [Online]. Available: <https://spark.apache.org/docs/latest/ml-classification-regression.html> [Accessed: Apr. 7, 2026].
- [3] Apache Spark. “VectorAssembler — PySpark 4.1.1 documentation,” Apache Spark. [Online]. Available: <https://spark.apache.org/docs/latest/api/python/reference/api/pyspark.ml.feature.VectorAssembler.html> [Accessed: Apr. 7, 2026].
- [4] Apache Spark. “pyspark.sql.Session — PySpark 4.1.1 documentation,” Apache Spark. [Online]. Available: <https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.Session.html> [Accessed: Apr. 7, 2026].
- [5] Apache Spark. “LogisticRegression — PySpark 4.1.1 documentation,” Apache Spark. [Online]. Available: <https://spark.apache.org/docs/latest/api/python/reference/api/pyspark.ml.classification.LogisticRegression.html> [Accessed: Apr. 7, 2026].
- [6] Apache Spark. “StandardScaler — PySpark 4.1.1 documentation,” Apache Spark. [Online]. Available: <https://spark.apache.org/docs/latest/api/python/reference/api/pyspark.ml.feature.StandardScaler.html> [Accessed: Apr. 7, 2026].
- [7] Apache Spark. “RandomForestClassifier — PySpark 4.1.1 documentation,” Apache Spark. [Online]. Available: <https://spark.apache.org/docs/latest/api/python/reference/api/pyspark.ml.classification.RandomForestClassifier.html> [Accessed: Apr. 7, 2026].
- [8] Apache Spark. “NaiveBayes — PySpark 4.1.1 documentation,” Apache Spark. [Online]. Available: <https://spark.apache.org/docs/latest/api/python/reference/api/pyspark.ml.classification.NaiveBayes.html> [Accessed: Apr. 7, 2026].
- [9] Apache Spark. “MulticlassClassificationEvaluator — PySpark 4.1.1 documentation,” Apache Spark. [Online]. Available: <https://spark.apache.org/docs/latest/api/python/reference/api/pyspark.ml.evaluation.MulticlassClassificationEvaluator.html> [Accessed: Apr. 7, 2026].